automatic vs manual parallelization

ideal scenario

- workflow where the programmer relies entirely on software tools (like a compiler) to convert a standard sequential program into a parallel one
- the process follows these steps
  - **sequential algorithm** → the programmer first designs a traditional, step-by-step algorithm
  - **sequential high level code** → this algorithm is then implemented using a standard sequential programming language
  - **automatic parallelization** → a tool, typically a compiler, analyzes the sequential code to find opportunities for parallelism
    - ex. independent instructions or loops
  - **parallel assembly implementation** → the tool automatically generates the low-level achine code that can run on multiple processors or execution units in parallel
- the programmer writes a simple sequential program and leaves the complex task of parallelization entirely on automatic tools
  - what kind of parallelism these tools can find?
    - **compiler's role** as Instruction-Level Parallelism (ILP) ehere the hardware executes multiple indpendent instructions from the same instruction stream at the same time
    - **vectorization (SIMD)** → single instruction is applied to multiple pieces of data at once
      - this is often inferred by analyzing loops that have no dependencies between iterations

l'ideale sarebbe che il compilatore facesse tutto da solo (in maniera automatica) ma spesso fallisce

automatic parallelization

why automatic parallelization fails?

```c
void and_or(int SIZE, int * and, int * or, int * b, int * c) {
    int i = 0;

    for(i=0; i<SIZE; i++)
        and[i] = b[i] & c[i];

    for(i=0; i<SIZE; i++)
        or[i] = b[i] | c[i];
}
```

- the two for loops in the and_or function look independent
  - one loop performs a bitwise AND operation
  - the second loop performs a bitwise OR operation
  - they could run in parallel

attenzione: si verifica una race condition solo nel caso in cui *and e *or puntano allo stesso indirizzo di memoria in questo caso entrambi i cicli calcolano il loro risultato e il valore finale dipenderebbe unicamente da quale ciclo ha completato l'ultima scrittura → non deterministico

problem: use of unrestricted C pointers because of the compiler cannot determine if the memory regions pointed to by and, or, b and c are distinct of if they overlap

- a programmer could call this function in a way that makes the pointers refer to the same or overlapping memory locations
  - ex. with and_or(9, &array_a[1], &array_a[1], array_b, array_a); the and or pointers point to the exact same memory location
- if and or or overlap, a data dependency exists between the two loops
  - **output dependency** (also known as Write-After-Write) → both loops write to the same memory location
  - running in parallel would create a **race condition**
    - the final value in the overlapping memory would depend on which loop's write operation finished last
    - this is unsafe and would produce an incorrect result
- the programmer or designer might know they will never call the function with overlapping pointers, but the compiler does not know this
  - the compiler must assume the worst-case scenario (that the pointers do overlap)
    - because it cannot prove that parallelization is safe, it will choose not to parallelize the code at all
    - we will have a correct but **sequential** program

se il compilatore non sa se due puntatori puntano alla stessa area di memoria non può parallelizzare per sicurezza, motivo per cui esegue il codice in maniera seriale

perché il compilatore considera il caso peggiore?

1. Il Compilatore deve coprire "tutti i casi possibili", non solo il tuo

Quando il compilatore trasforma la tua funzione in codice macchina, deve creare una versione che funzioni **sempre**, per qualsiasi chiamata futura, non solo per quella che hai scritto tu in quel momento.

Immagina di avere questa funzione (tratta dalle slide): `void and_or(int *and, int *or, ...)`

Anche se tu nel tuo `main` la chiami correttamente con due array diversi: `and_or(..., Array4, Array5)`

Il compilatore deve garantire che il codice macchina generato per `and_or` sia sicuro anche se un utente "malizioso" o distratto scrivesse:

`and_or(..., Array4, Array5).`

In questo secondo caso ("caso pessimo"), i due puntatori `and` e `or` punterebbero alla stessa memoria (aliasing). Se il compilatore avesse parallelizzato il codice, due thread scriverebbero contemporaneamente nella stessa cella, creando dati corrotti. Poiché il compilatore non può vietare questo uso "sbagliato" a priori, per sicurezza deve generare un codice sequenziale (lento ma sicuro).

2. Il problema della "Visibilità" (Compilazione Separata)

Spesso il codice della funzione e il codice che la chiama (il `main`) si trovano in file diversi o addirittura in librerie diverse. Quando il compilatore lavora sul file della funzione, **non vede** come verrà chiamata. Vede solo `int *a, int *b`. Non ha idea se `a` e `b` saranno collegati a variabili diverse o alla stessa variabile. Essendo "cieco" sul contesto esterno, deve assumere lo scenario peggiore (che puntino alla stessa cosa).

3. I nomi delle variabili non contano per la memoria

Per il compilatore, i nomi diversi (`Array4`, `Array5`) sono solo etichette per noi umani. A livello hardware, ciò che conta sono gli **indirizzi di memoria**. È perfettamente legale in C/C++ avere due puntatori con nomi diversi che contengono lo stesso indirizzo numerico (es. `int *p = &x; int *q = &x;`). Il compilatore sa che questa possibilità esiste sempre, quindi non può fidarsi solo dei nomi diversi.

- a fully "hands-off" approach is not feasible
  - key reasons
    - the program was originally designed to be executed sequentially
      - a compiler's tools are not powerful enough to extract all tha available parallelism from a specification that wasn't built with parallelism in mind
    - a compiler must be conservative
      - if it cannot prove that parallelization is safe, it will choose to generate correct sequential code rather than risk an incorrect parallel program
  - the failur of complete automatic parallelization highlights a fundamental concept in high-performance computing
    - **amdahl's law**
      - this law states that the maximum speedup of a program is limited by the portion of it that must be executed sequentially
      - automatic tools are often unable to minimize this sequential part because they cannot fundamentally rethink the algorithm's structure
    - **parallel-first design (PCAM)**
      - effective parallel programming usually requires designing the algorithm for parallelism from the very beginning
      - the **PCAM (Partitioning, Communication, Agglomeration, Mapping) methodology**, for example, lists Partitioning (breaking the problem into parallel tasks) as the first step
        - the opposite of the "automatic" workflow, which tries to find parallelism at the very end
- since fully automatic parallelization is not feasible, the task falls to the programmer
  - **parallelization by hand**

parallelization by hand

there are three critical challenges that a programmer must manage when **parallelizing code by hand**

- type of parallelism
  - the programmer must first identify what kind of parallelism is available in the problem
  - **bits**
    - this involves performing operations on the individual bits of a data word (like an int) simultaneously
      - the lowest level of parallelism
    - core concept
      - a standard data word (like a 32-bit or 64-bit integer) can be treated as a collection of separated data points
        - ex. 32 or 64 individual boolean values
    - how it works?
      - a single CPU instruction (like a bitwise AND, OR, or XOR) operates on all bits in the word simultaneously
        - ex. a single OR instruction can perform 32 or 64 logical OR operations in one clock cycle
    - hardware revelance
      - this form of parallelism is fundamental in hardware design (ASICs, FPGAs), where logic is built directly from gates that operate on bits

      implementato direttamente nella circuiteria dell'ALU del processore → l'ampiezza del bus dati e dei registri determina quanto parallelismo a livello di bit è possibile
    - software revelance
      - using bitwise operations, a programmer can perform complex set operations (like union or intersection) on dozens of elements with a single instruction

    il parallelismo a livello di bit si basa sull'aumento della dimensione della word size del processore:
    - **con i processori a 8-bit, per spostare 64 bit dovevamo aspettare 8 cicli**
    - **con i processori a 64-bit, per spostare 64 bit è sufficiente un solo ciclo**

    PARALLELISMO: all'interno del processore, le operazioni su tutti i 64 bit avvengono simultaneamente nello stesso istante e non in sequenza

    2. Come funziona tecnicamente (L'esempio della Somma)

    Le slide (pag. 7) usano l'esempio della somma per spiegare questo concetto.

    Immagina di dover sommare due numeri molto grandi (a 32 bit).

    $$A + B = C$$

    - **Senza parallelismo a livello di bit (es. Processore a 8 bit):** Il processore non "vede" il numero intero. Deve spezzarlo in 4 pezzetti da 8 bit.
      1. Somma i primi 8 bit (e segna il riporto).
      2. Somma i secondi 8 bit + il riporto precedente.
      3. Somma i terzi 8 bit + il riporto.
      4. Somma gli ultimi 8 bit + il riporto. *Risultato:* Servono **4 istruzioni** separate.
    - **Con parallelismo a livello di bit (es. Processore a 32 bit):** Il processore ha circuiti abbastanza larghi da contenere tutto il numero.
      1. Prende tutto A e tutto B.
      2. Li somma in un colpo solo. *Risultato:* Serve **1 sola istruzione**.
  - **instructions**
    - this involves performing operations on the individual bits of a data word (like an int) simultaneously
      - the lowest level of parallelism
    - how it works?
      - modern processors have sophisticated hardware designed to find and execute independent instructions at the same time
    - hardware support
      - enabled by features like
        - **multiple execution units**
          - a core may have several Arithmetic Logic Units (ALUs), allowing it to perform multiple calculations (e.g., an add and a multiply) at the same time
        - **pipelining**
          - the processor breaks down the execution of an instruction (e.g., fetch, decode, execute, write-back) into stages, allowing multiple instructions to be in different stages of execution at once
        - **SIMD units**
          - Single Instruction, Multiple Data units execute one instruction on a large vector of data (e.g., adding four pairs of numbers simultaneously).

      - il processore ha più unità di esecuzione che possono lavorare in parallelo
      - il processore può cambiare l'ordine delle istruzioni per tenere occupate le unità di esecuzione, purché il risultato finale non cambi

      in pratica, i diversi processori possono essere in fasi diverse allo stesso momento → mentre un'istruzione viene eseguita, la successiva viene decodificata e quella dopo viene prelevata dalla memoria così ogni processore fa sempre un'azione senza perdere alcun tempo
    - extraction
      - this type of parallelism is typically extracted automatically by the compiler and the processor hardware
        - the hardware itself finds these small-scale opportunities to run instructions in parallel

      ex. nella funzione and_or vista prima, il compilatore potrebbe srotolare il ciclo o usare istruzioni vettoriali per elaborare più iterazioni insieme

    Richieste Hardware

    Per fare questo salto (da 8 a 64 bit), l'hardware deve essere fisicamente più grande e complesso:

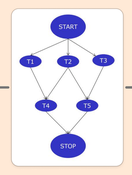
    1. **ALU (Arithmetic Logic Unit) più larga:** L'unità che fa i calcoli deve avere fisicamente 64 "corsie" parallele di transistor per gestire i 64 bit insieme.
    2. **Bus Dati più largo:** Le "autostrade" che trasportano i dati dalla memoria al processore devono avere 64 corsie, altrimenti il processore aspetterebbe i dati a pezzi.
    3. **Registri più grandi:** Le "tasche" interne del processore dove appoggia i dati momentanei devono essere grandi 64 bit.

    Richieste Software

    Qui c'è una distinzione importante rispetto ad altri tipi di parallelismo:

    - **Trasparenza:** Spesso il programmatore non deve fare nulla di speciale nel codice sorgente (scrivi `c = a + b` e basta).
    - **Il Compilatore:** È lui che deve sapere che sta lavorando su una macchina a 64 bit. Se compili il codice per un vecchio processore a 8 bit, il compilatore "spezzerà" la tua somma in 8 mini-somme. Se compili per una macchina a 64 bit, userà l'istruzione singola.
  - **tasks**

    si tratta di suddividere il problema in compiti distinti e farli eseguire contemporaneamente, su processori o core diversi
    - this involves identifying logically distinct sections of computational work (functions or large loops) that can be executed concurrently by different processors
      - typically a program or program-like set of instructions that is executed by a processor
        - Thread-Level Parallelism (TLP)
      - we have a **parallel program** if multiple tasks tunning on multiple processors
    - extraction
      - it must be explicitly designed by the programmer
        - this type of parallelism is usually difficult to extract automatically by a compiler
    - how to visualize task parallelism

      - the **vertices** represent the tasks themselves
      - the **edge** represent dependencies (precedencies) or the flow of data between tasks
      - directed acyclic graph (DAG)
        - T1, T2 and T2 can all run in parallel after START
        - T4 must wait for both T1 and T2 to finish
        - T5 must wait for both T2 and T3 to finish
    - pipeline model
      - this is a different model of task parallelism common in streaming applications
        - tasks are arranged in a linear chain, like a factory assembly line or a processor pipeline
        - data passes from one task to the next (P1 → P2 → P3 → P4)
      - **execution**
        - this model is ideal for streaming data (like audio or video encoding)
        - each task run repeatedly, performing its "stage" of work on each piece of data that flows through the pipe
    - how tasks exhange the data that the graph edges represent?
      - **shared memory**
        - all processors and tasks share a single, global memory address space
        - communication is implicit
          - Task A writes a value to a memory location
          - Task B reads it from that same location
        - this is the model used by technologies like **OpenMp** and **Pthreads**
          - it requires synchronization (like locks) to prevent multiple tasks from writing to the same location at the same time (a **race condition**)
      - **message passing**
        - each task has its own private memory that no other task can access
        - communication is explicit
          - Task A must package its data into a "message and actively send it to Task B
          - Task B must actively receive it
        - this is the model used by technologies like **MPI (Message Passing Interface)**
          - very common in large supercomputers and clusters because it doesn't require expensive shared memory hardware
  - ex. if memory locations do not overlap (restrict) we can extract parallelism at different levels

    restrict serve per fare una promessa al compilatore → stiamo garantendo che i puntatori and, or, b and c puntano a zone di memoria separate e non sovrapposte

    ```c
    void and_or(int SIZE, int * restrict and, int * restrict
      or, int * restrict b, int * restrict c){
      int i = 0;
      for(i=0; i<SIZE; i++)
        and[i] = b[i] & c[i];
      for(i=0; i<SIZE; i++)
        or[i] = b[i] | c[i];
    }
    ```

    - **tasks** (inter-loop): the two for loops are independent and can be run in parallel

      i due cicli non si influenzano a vicenda → possiamo assegnarli a core che lavorano contemporaneamente su "compiti" diversi ma all'interno della stessa funzione
    - **instruction** (intra-loop): each iteration of a single loop is independent
      - ex. calculating and[0] has no effect on and[1]
      - the compiler can unroll the loop or use SIMD instructions to execute multiple iterations at once

      la CPU può caricare un "vettore" di dati ed eseguire l'AND su tutti e 4 in un colpo di clock (SIMD)
    - **bit** (intra-operation): the bitwise & and | operations are themselves parallel, as all 32 or 64 bits are computed in a single clock cycle

      quando la CPU esegue b[i] & c[i], non fa l'AND su ogni singolo bit, ma poiché l'ALU possiede circuiti dedicati per eseguire l'operazione su tutti i 32 bit esattamente nello stesso istante
- algorithm design
  - the programmer must decide how to structure the parallel algorithm
    - **parallelize existing algorithm** → take a current sequential algorithm and try to modify it for parallel execution
    - **design from scratch** → design a new algorithm that is inherently parallel form the very beginning
  - the parallelism in your algorithm must match the parallelism offered by your hardware
    - we need to consider which parallelism is available on the considered architecture
      - non suitable parallelism can introduce
    - we need to "describe" the parallelism to the compilation tools to make it exploitable
- provide information to tools
  - the programmer must use a specific programming model, library or language extension to explicitly tell the compiler and hardware how the code shold be parllelized